Algorithm and Data Structure 1
Binary Search Trees
Deadline: See Moodle

Python
Institute of Pervasive Computing
Compiled: February 28, 2026

JOHANNES KEPLER UNIVERSITY LINZ

Elaboration Time: Remember the time you need for the elaboration of this assignment and document it in the Moodle feedback.
For this assignment, please submit a zip containing:

- The solution of the pen & paper exercise.
- The source code of your `bst.py`.

Note: Don't change the signature of any method that is given in the code skeleton (including constructors). Ensure, that all unit test run without any compilation errors. You may add auxiliary methods and classes and reuse code where possible. Remember to add all code-files you changed or created in the submission zip.

Assignment Part	Points
Binary Search Tree – Insert and Remove	6
Binary Search Tree – Implementation	18
Total:	24

Scanned with TapScanner

1. **Binary Search Tree – Insert and Remove** ..... **6 Points**

For this exercise the following binary search tree (BST) is given:

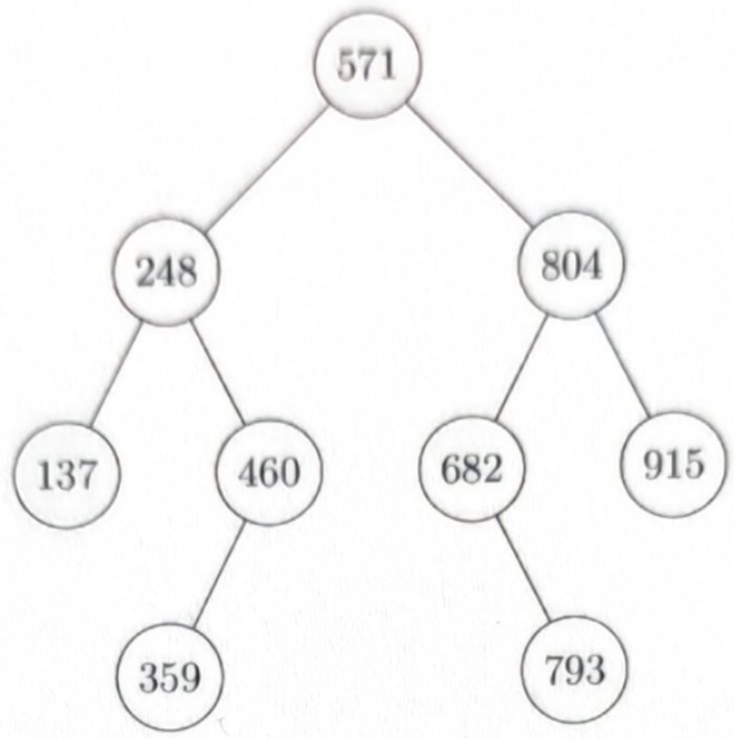


Figure 1: Binary Search Tree (BST)

Apply the following operations in this order:

k125 ~~k7~~ 651

(a) (2 points) **Insert** a new node with the number formed by the last three digits of your student ID. Example: For the student ID k24680**123**, the inserted node should use the key **123** (leading zeros are ignored). Describe the required comparisons and draw the BST after the insertion.

(b) (2 points) **Remove** the root node (with the key **571**) from the BST after you inserted the new node from (1). Explain what changes have to be applied to the BST, e.g., which node becomes the new root node, how to rearrange the old parent-child-connections to maintain the order, etc. Draw the BST after the removal.

(c) (2 points) **Insert** a new node with the number formed by the first three digits of your student ID. Example: For the student ID k**246**80123, the inserted node should use the key **246** (leading zeros are ignored). Describe the required comparisons and draw the BST after the insertion.

1. a) insert 651

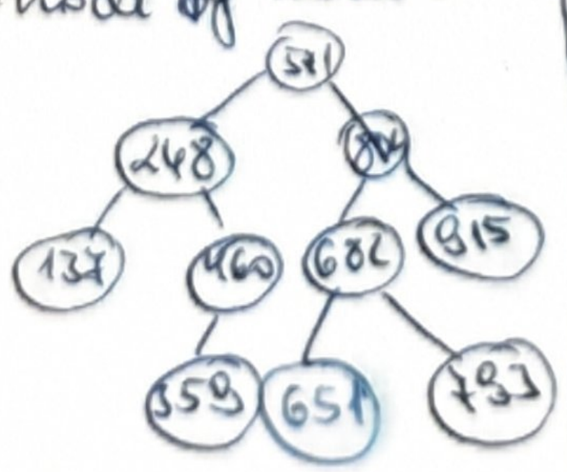
comparison:

651 > 571 - go right

651 < 804 - go left

651 < 682 - go left

Therefore 651 is inserted as a left child node of node 682.



1b) remove the root (571)

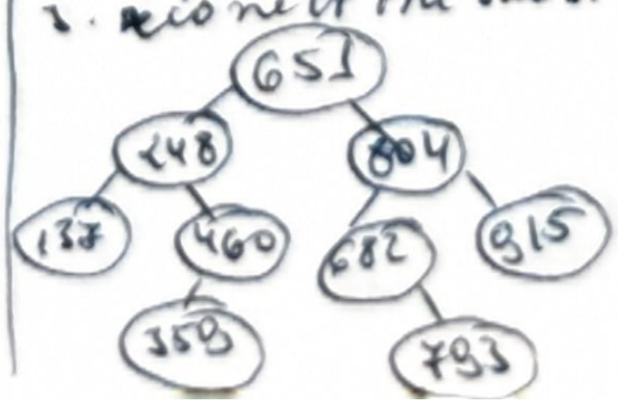
the root 571 has two children, so we replace it whit the smallest node from the right subtree.

Right subtree:

804 → left child 651

the smallest node is 651

1. Replace 571 whit 651
2. Remove the original node 651 from the position
3. Reconnect the subtree properly



1c. insert 125

comparison:

125 < 651 - go to left

125 < 248 - go to left

125 < 137 - go to left

Therefore 125 is inserted as a left child node of 137

Scanned with TapScanner

2. **Binary Search Tree – Implementation ..... 18 Points**

   (a) (18 points) Implement a binary search tree. Fill in the missing parts marked with `# TODO` comments. Your goal is defined by the documentation and comments.

   Files to implement:

   - `bst.py`

   Test your solution with the following tests:

   - `test_bst.py`

Scanned with TapScanner

Notes on `runtime_comparison()`: This method shall create a BST from a given linear list (**without duplicates**) and analyze the runtime for the search of a key. For this determine the **number of comparisons** needed to find the key in both, the BST (key comparison + comparison for branch decision) and the linear list (key comparison only).

Example on how to count the comparisons in `[8, 17, 10, 3, 1]`: The list would result in the following Binary Search Tree (Figure 2):

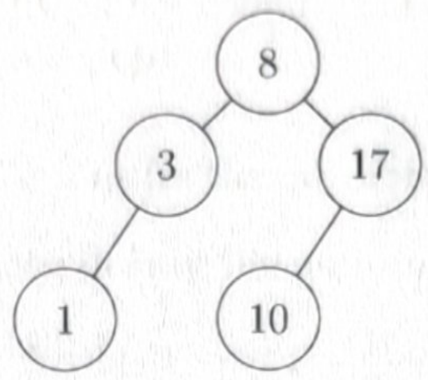


Figure 2: Binary Search Tree (BST)

Searching for key '3' requires 4 comparisons for the list:

1. `3 != 8`
2. `3 != 17`
3. `3 != 10`
4. `3 == 3`

For the BST, it would require 3 comparisons:

1. `3 != 8`
2. **`follow left branch`**
3. `3 == 3`

The result needs to be returned in form of a `tuple[int, int]` with the number of comparisons needed for the list as its first value, and the number of comparisons needed for the BST as its second value.

Scanned with TapScanner

Note on unit tests: In the unit test file, you will find the test case: `test_runtime_comparison_with_pregen_list()`, that tests the structures with large lists by searching for the last inserted element (= worst case for linear list). You should see a significantly better performance of the BST here.

For these tests a pregenerated sequence of random numbers stored in the text file `testListWithoutDuplicates.txt` (which is provided in the skeleton) is used. Depending on how you organize your project and where you place that text file, it can happen that these unit tests fail, because the file is not found. Then you probably have to adjust the path to that text file, following these steps:

- Within the mentioned test case look for the call of the function: `create_list_from_file()`.
- The argument of that function defines the path to the text file as a String.
- You can replace that String by an absolute path pointing to where you stored the text file, e.g.: If your test file is in "`/temp/`", then the line with the call of `create_list_from_file` could look like:

```
... = create_list_from_file("/temp/testListWithoutDuplicates.txt")
```


Scanned with TapScanner